Ders 3 Genişletilmiş Notu: Dizi Tanıyıcı, Saklayıcılar ve Mikro İşlemler, Seri Toplayıcı, Asenkron ve Senkron Sayıcılar

Ders 3 Genişletilmiş Notu: Dizi Tanıyıcı, Saklayıcılar ve Mikro İşlemler, Seri Toplayıcı, Asenkron ve Senkron Sayıcılar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Geçen Haftanın Tekrarı: Mealy ve Moore Modelleri
- 2. Ardışıl Devre Tasarımı: Dizi Tanıyıcı
- 3. Saklayıcılar
- 4. Saklayıcı Transfer İşlemleri ve Mikro İşlemler
- 5. Seri Toplayıcı
- 6. Asenkron (Ripple) Sayıcılar
- 7. Senkron İkili Sayıcı Tasarımı
- 8. Load Girişli Sayıcıdan Başka Sayıcılar Elde Etmek
- 9. Rastgele Sayma Düzenli Sayıcı ve JK Geçiş Tablosu

1. Geçen Haftanın Tekrarı: Mealy ve Moore Modelleri

Dönemin üçüncü haftasında hoca, geçen hafta görülen kombinasyonel ve ardışıl devreleri hatırlatarak başladı. Flip-flop'larla yapılan ilk çalışma bir tersine mühendislik (reverse engineering) çalışmasıydı: Bir devre şeması verilmiş ve bu devrenin ne iş yaptığı bulunmaya çalışılmıştı. İzlenen sıra şöyleydi:

- Her flip-flop'un giriş denklemi yazıldı. Devrede iki flip-flop, dolayısıyla iki durum değişkeni (state variable) vardı.
- Devrenin X girişine ve durum değişkenlerine bağlı Y çıkışının denklemi yazıldı.
- Bu denklemlerden bir durum tablosu (state table) oluşturuldu.
- Durum tablosundan yararlanılarak durum diyagramı (state diagram) çizildi.

1.1 Mealy Modeli

Bu devre bir Mealy modeli devreydi. Çünkü Y çıkışı yalnız durumlara değil girişe de bağlı olarak değişiyordu: Çıkış fonksiyonu, $A + B$ ifadesiyle birlikte X'in değilini de içeriyordu, yani $Y = (A + B)\bar{X}$. Çıkış fonksiyonunda giriş etkiliyse bu tür devrelere Mealy modeli devre denir.

1.2 Moore Modeli

Moore modeli devrede ise çıkış yalnız durumlara bağlı olabilir. Örnekteki devrede bir flip-flop, iki giriş (X ve Y) ve bir Z çıkışı vardı. Z çıkışı A durum değişkenine bağlıydı. Durum tablosuna bakıldığında Z çıkışının, A'nın mevcut durumuyla (present state) aynı olduğu görülüyordu. Bu nedenle devreye Moore modeli denir.

İki modelin durum diyagramları arasındaki tek fark çıkışın yazıldığı yerdir:

	Mealy	Moore
Çıkış neye bağlı?	Durumlara ve girişe	Yalnız durumlara
Çıkış diyagramda nerede?	Okların üzerinde, girişle birlikte	Durumu gösteren dairenin içinde, durumla birlikte
Okların üzerinde ne var?	Giriş ve çıkış	Yalnız giriş

> **Hocanın vurgusu:** Bu iki yapıyı birbirinden ayırt edebilmeniz gerekir. Farkı hem durum diyagramından hem de devrenin çıkış fonksiyonunu yazarak görebilirsiniz.

2. Ardışıl Devre Tasarımı: Dizi Tanıyıcı

Hoca ardışıl devre tasarımı için öğrencilerle birlikte bir örnek çözdü. Amaç, gelen girişin içinde belirli bir sayı dizisini (sequence) bulan bir devrenin, yani dizi tanıyıcının (sequence recognizer) durum diyagramını oluşturmaktı.

2.1 Problem

Devre bir blok olarak düşünülür. Bir bitlik bir X girişi ve bir Z çıkışı vardır. Başta flip-flop'ların doğrudan (direct) reset girişlerine bir reset işareti uygulanır. Bu giriş clock'tan bağımsızdır ve bütün flip-flop'ları sıfırlar. İlk durum buradan başlar.

Devrenin amacı X üzerinde 1, 1, 0, 1 dizisinin geçtiğini $Z = 1$ yaparak bildirmektir. Önceki üç giriş 1, 1, 0 ve şimdiki giriş 1 ise $Z = 1$ olur, aksi durumda $Z = 0$ olur.

2.2 Örnek Giriş ve Çıkış

X girişini bir zaman dizisi (time sequence) gibi düşünmek gerekir. Devre, gelen her clock işaretiyle X'i örnekler ve buna göre bir Z çıkışı üretir. Hocanın tahtadaki örneği şöyledir:

Giriş sırası	1	2	3	4	5	6	7	8	9	10	11
X	1	0	1	1	0	1	1	0	1	0	0
Z	0	0	0	0	0	1	0	0	1	0	0

İlk beş girişte aranan dizi henüz görülmediği için çıkış 0'dır. Altıncı girişte son 1 geldiğinde dizi yakalanır ve Z çıkışında 1 üretilir. Bir sonraki adımda pencere kaydırılınca son dört giriş 1, 0, 1, 1 olur. Bu aranan dizi değildir, çıkış 0'dır. Sekizinci girişten sonra gelen 1 ile yine 1, 1, 0, 1 görülür. Burada ilk dizinin son 1'i, ikinci dizinin ilk 1'i olarak da kullanılır. Hocaya göre böyle bir örtüşme (overlap) olsa bile yeni bir blok görülmüştür ve devre 1 üretmelidir. Ardından gelen 0'lar için çıkış yine 0'dır.

2.3 Durum Diyagramının Kurulması

Tasarım için önce durum diyagramı gerekir. Değişken sayısı henüz bilinmediği için durumlara harf verilir. Başlangıç durumu A'dır ve devre reset işaretiyle buraya gelir. Her durumun bir anlamı vardır:

- **A:** Aranan dizinin hiçbir elemanı yakalanmadı.
- **B:** Girişte bir tane 1 görüldü.
- **C:** Arka arkaya iki tane 1 görüldü.
- **D:** Arka arkaya 1, 1, 0 görüldü.

Geçişler bu anlamlardan çıkarılır:

- A'dayken 0 gelirse dizinin hiçbir elemanı yakalanmamıştır, çıkış 0 olur ve A'da kalınır. 1 gelirse dizinin birinci elemanı yakalanmıştır, B'ye geçilir ve çıkış 0 olur.
- B'deyken 0 gelirse başa dönülür, çünkü dizinin ikinci elemanı 1 olmalıdır. İkinci bir 1 gelirse C'ye geçilir.

- C'deyken 0 gelirse dizinin üçüncü elemanı da bulunmuştur, D'ye geçilir.
- D'deyken 0 gelirse A'ya dönülür ve çıkış 0 olur.
- D'deyken 1 gelirse dizi tamamlanmıştır, $Z = 1$ üretilir ve B'ye gidilir.

Soru: C'deyken 1 gelirse ne yapılmalı? Bir öneri A'ya dönmekti. **Cevap:** C üzerinde dönmek yeterlidir. C'de arka arkaya iki tane 1 görüldüğü garanti edilmiştir. Bir 1 daha gelince pencere kaydırıldığında son iki giriş yine 1, 1 olur. Dolayısıyla C'nin anlamı bozulmaz.

Soru: D'deyken 1 gelirse neden A'ya değil B'ye gidiliyor? **Cevap:** Çünkü B'de olmanın anlamı "bir tane 1 yakaladım" demektir. Bir dizinin son 1'i bir sonrakinin başlangıç 1'i olabilir, örnekte de bu durum görülmüştü.

Diyagramın tablo hâli (geçişler Sonraki durum / Z biçiminde):

Mevcut durum	$X = 0$	$X = 1$
A	A / 0	B / 0
B	A / 0	C / 0
C	D / 0	C / 0
D	A / 0	B / 1

2.4 Durum Tablosu ve Durum Kodlaması

Bir sonraki adım diyagramı durum tablosuna taşımak ve flip-flop'ların giriş fonksiyonlarını yazmaktır. Dört durum için iki durum değişkeni, yani iki flip-flop yeterlidir. Tabloya mevcut durum, X girişi, sonraki durum ve Z çıkışı yazılır.

Mevcut durumları yazarken A, B, C, D harflerine birer değer atamak gerekir. Hangi duruma hangi kodun verildiği devrenin karmaşıklığını etkiler. Hoca $A = 00$, $B = 01$, $C = 11$, $D = 10$ atamasını kullandı. Ona göre C ile D'nin kodları yer değiştirirse daha karmaşık bir flip-flop giriş fonksiyonu çıkar. Amaç işi daha az malzemeyle yapmak olduğu için bu seçim önem kazanır. Hoca, bilgisayar mühendisliği açısından bu kadar ayrıntılı devre çözümü gerekmediğini söyleyerek konunun üzerinde fazla durmadı. Sonraki durum ve Z sütunları elimizdeki durum diyagramından doldurulur:

Mevcut (Q_1Q_0)	X	Sonraki (Q_1Q_0)	Z
00 (A)	0	00	0
00 (A)	1	01	0
01 (B)	0	00	0
01 (B)	1	11	0
11 (C)	0	10	0
11 (C)	1	11	0
10 (D)	0	00	0
10 (D)	1	01	1

(ara adım eklendi: Tablo slaytta gösterildi, satırları altyazıda okunmuyor. Değerler derste kurulan diyagramdan ve hocanın kodlamasından yazıldı. Durum harfleriyle karışmasın diye durum değişkenlerine bu notta Q_1, Q_0 dendi.)

> **Ek bilgi (derste anlatılmadı):** Denklemler derste yazılmadı, yukarıdaki tablodan Karnaugh sadeleştirmesiyle $D_{Q_1} = Q_1Q_0 + Q_0X$, $D_{Q_0} = X$, $Z = Q_1\bar{Q}_0X$ çıkar. C ile D'nin kodları yer değiştirirse ($C = 10$, $D = 11$) $D_{Q_1} = \bar{Q}_1Q_0X + Q_1\bar{Q}_0$ ve $D_{Q_0} = \bar{Q}_1\bar{Q}_0X + Q_1\bar{Q}_0\bar{X} + Q_1Q_0X$ olur. Bu hesap, hocanın "tersi daha karmaşık çıkar" sözünü doğrular. Z çıkışı X'e bağlı olduğu için bu devre, Bölüm 1'deki tanıma göre bir Mealy modeli devredir.

3. Saklayıcılar

Haftanın esas konusu saklayıcılar (registers) ve sayıcılardır (counters). Hoca bunları hep bir bilgisayarın, bir işlemcinin içinde nasıl kullanacağımız açısından anlatacağını söyledi. Saklayıcılar üzerinde yapılan en küçük işlem parçasına mikro işlem (micro operation) denir. Önce saklayıcıların bir load enable ucuyla nasıl tasarlanacağına ve kaydırmalı saklayıcılara, sonra sayıcılara bakılacaktır.

Saklayıcı, ikili bir sayıyı depolamak için kullanılan en temel devre elemanıdır.

3.1 Basit Kaydırmalı Saklayıcı

İlk örnek, D flip-flop'larla kurulmuş basit bir kaydırmalı saklayıcıdır (shift register). Bütün D flip-flop'lar pozitif kenar tetiklemelidir (positive edge triggered). Clock girişleri senkron olarak birbirine bağlıdır ve devrenin tek bir clock girişi vardır.

Soldaki flip-flop'un bir seri giriş (serial input) ucu vardır. Buradan verilen değer, gelen her clock işaretiyle soldaki flip-flop'tan sağdakine aktarılır. Bunun sebebi, bir flip-flop'un Q çıkışının diğerinin D girişine bağlanmasıdır. En sağdaki flip-flop'un çıkışı seri çıkıştır (serial output).

Hoca öğrenilen her blok için bir standart grafik sembolü de verileceğini söyledi. Bu sembolde bloğun sol tarafında girişler, sağ tarafında çıkışlar bulunur. Örnekteki sembolde bir clock girişi, bir seri giriş, 4 bitlik yapı olduğunu gösteren işaret ve bir seri çıkış vardır.

3.2 Saklayıcı Tasarımında İki Yol

Slaytta saklayıcı tasarımı için iki yöntem geçer:

- **Hazır kombinasyonel devre eklemek:** Nasıl çalıştığı önceden bilinen bir devre yapısı saklayıcıya eklenir (add predefined combinational circuits to registers).
- **Bir bitlik hücreden başlamak:** Yapı baştan bir bit üzerinden tasarlanır, sonra bu hücreler bir araya getirilir. Bir bitlik hücrenin yalnız iki durumu vardır ve çıkış genellikle durum değişkenidir.

3.3 Load Girişi Eklemek: Clock Gating

Amaç, saklayıcıya yeni sayı yüklemeye yarayan bir load girişi eklemektir. Saklama ya da yükleme işlemi bir işaretle kontrol edilmelidir. Load = 1 iken veri girişlerindeki değer saklayıcıya yüklenir. Yeni veri yüklenmeyecekse saklayıcı o andaki değerini korur.

Birinci çözüm clock gating'dir. Dört D flip-flop'lu yapının clock girişlerinin önüne küçük bir devre eklenir. Load işareti bir inverter'dan geçer, sonra clock ile birlikte bir OR kapısına girer. Bu OR kapısının çıkışı bütün flip-flop'ların clock girişine gider.

- **Load = 1:** Inverter çıkışı 0 olur, OR kapısı açık hâle gelir. Clock işareti çıkışa, yani bütün flip-flop'ların clock girişine aktarılır. Yükleme, clock'un pozitif kenarlarında gerçekleşir.
- **Load = 0:** Inverter çıkışı 1 olur ve OR kapısını kilitler. Çıkışta clock'un değişimleri görülmez, sürekli 1 görülür.

Yani load girişi, clock işaretinin devreye ulaşıp ulaşmayacağını seçen bir uç gibi davranır.

Bu çözümün dezavantajı, işlemcinin içindeki eşzamanlılığı bozmasıdır. Bir CPU'da birden fazla saklayıcı, sayıcı ve başka eleman bulunur. Clock ile çalışan bütün birimler senkron hareket etmeli, hepsi clock'un aynı pozitif kenarında bir durumdan diğerine geçmelidir. Oysa burada clock yoluna iki kapı (bir inverter ve bir OR) eklenmiştir ve bu kapıların belli bir yanıt verme süresi vardır. Bu saklayıcı clock işaretini gecikmeli alır.

Soru: Inverter gecikme yaratıyorsa neden load kısmına NOT kapısı bağlıyoruz, kaldıramaz mıyız? **Cevap:** Amaç load = 1 iken yükleme yapmaktı, o yüzden kullanıldı. NOT kapısı kaldırılırsa devre load = 0 iken çalışır. Çözüm değiştirilebilir: OR yerine AND kapısıyla tek kapılı bir çözüm de yapılabilir. Fakat her durumda en az bir kapı kullanılır ve sorun budur. Kullanılan kapı yüzünden bu clock ile sistemin clock'u arasında gecikme oluşur. O clock girişi başka saklayıcıların ve sayıcıların da clock girişidir ve hepsi aynı anda durum değiştirecektir. Araya

eleman eklenirse çıkış gecikir, onunla senkron çalışan diğer devre görmesi gereken gerçek değeri göremez.

> **Hocanın vurgusu:** İki çözüm arasındaki fark gerçekten önemlidir. Senkron tasarımlarda clock girişi kesinlikle ekstra bir kapı kullanılmadan bağlanmalıdır. Clock gating bağımsız çalışacak bir devrede kullanılabilir, ama birbiriyle senkron çalışan onlarca birimi bir araya getirirken tercih edilmez.

3.4 Load-Controlled Feedback

İkinci çözüm, clock'un önüne değil D flip-flop'un D girişinin önüne bir devre yerleştirir (registers with load-controlled feedback). İstenen iki durum vardır:

- Load = 1 iken In_i girişleri flip-flop'lara yüklenir.
- Load = 0 iken clock gelmeye devam ettiği hâlde flip-flop içindeki değeri muhafaza eder (hold current value).

Hoca, en düşük anlamlı bitin giriş denklemini yalnız bu tanımdan yazdı. Flip-flop'un çıkışı A_0, load ucu L ile:

$$D_{A_0} = \bar{L}\, A_0 + L\, \text{In}_0$$

Bu denklem iki AND ve bir OR kapısıyla kurulur. Load = 0 iken birinci AND kapısı çıkışı flip-flop'a geri verir, load = 1 iken ikinci AND kapısı In_0'ı geçirir. OR kapısı bu toplamı alıp D girişini oluşturur. Saklayıcının değerini koruması, flip-flop'un çıkışını yeniden girişine bağlayıp aynı değeri yükleyerek sağlanır.

Bu çözümde clock işareti bütün flip-flop'lara doğrudan uygulanır, araya bir kapı eklenmez. Böylece blok işlemcideki diğer bloklarla senkron çalışabilir. Bu nedenle tercih edilen çözüm budur.

Slaytta bu devre önce tek bir hücre için, sonra dört hücreye uygulanmış olarak gösterilir. Standart grafik sembolünde D girişi, enable ucu, clock girişi ve Q çıkışı bulunur. Bloğun içinde bu devrenin olduğu bilinir.

3.5 Shift ve Load Kontrollü Saklayıcı

Bir sonraki örnekte saklayıcıya bir de shift kontrol ucu eklenir ve fonksiyon tablosu verilir:

Shift (S)	Load (L)	İşlem
0	0	Değişmez (no change, hold current value)
0	1	Paralel veri yükleme (load parallel data)
1	X	Kaydırma (Q_0'dan Q_3'e doğru)

Shift = 1 iken başka bir fonksiyon olmadığı için load ucu don't care bırakılmıştır.

Slayttaki işlem "shift down" diye adlandırılmıştır. Hoca bu adı doğru bulmadı. 4 bitlik sayı ağırlıklarıyla birlikte soldan sağa $Q_3Q_2Q_1Q_0$ biçiminde yazılır: Q_3 en yüksek anlamlı, Q_0 en düşük anlamlı bittir. Q_0'dan Q_3'e doğru kaydırmanın yönü soladır.

> **Hocanın vurgusu:** Shift down, shift up yerine assembly'den ve mikroişlemcilerden bilinen shift left, shift right adlarını kullanmak daha doğrudur. Slayttaki işlem bir shift left işlemidir.

Flip-flop giriş denklemi fonksiyon tablosuna bakılarak yazılır. D girişleriyle karışmasın diye paralel veri girişlerine Data_i denir. Tablonun her satırı denklemde bir terim verir. Üçüncü satırda load önemsiz olduğu için L yazılmaz. Shift left işleminde Q_0'ın yerine dışarıdan seri giriş (SI) gelir:

$$D_0 = \bar{S}\bar{L}\, Q_0 + \bar{S}L\, \text{Data}_0 + S\, \text{SI}$$

Soru: Aynı denklem D_1 için yazılırken shift = 1 iken giriş nereden gelir? Bir cevap seri giriş oldu. **Cevap:** Hayır, Q_0'dan gelir. Çünkü sola kaydırmada Q_1'in yerine Q_0 gelir:

$$D_1 = \bar{S}\bar{L}\,Q_1 + \bar{S}L\,\text{Data}_1 + S\,Q_0$$

Devrede her D girişinin önünde bu üç terimi sağlayan 3 AND kapısı bulunur. Verilen fonksiyon tablosundan devrenin iç yapısı bu şekilde kurulur.

3.6 MUX ile Çok Fonksiyonlu Saklayıcı

Aynı iş ayrı kapılar yerine multiplexer ile de yapılabilir. Bu örnekte fonksiyon tablosunda iki bitlik mod kontrol girişi (S_1S_0) vardır. Slaytta yine shift down ve shift up yazılıdır. Hoca bunları sevmediğini söyleyip shift down'ın shift left, shift up'ın shift right demek olduğunu belirtti.

S_1S_0	İşlem	D_i	MUX girişi
00	Değişmez	Q_i	0
01	Shift left (slaytta shift down)	Q_{i-1}	1
10	Shift right (slaytta shift up)	Q_{i+1}	2
11	Paralel yükleme	Data_i	3

Tablodan i. flip-flop'un giriş denklemi yazılır:

$$D_i = \bar{S}_1\bar{S}_0\,Q_i + \bar{S}_1S_0\,Q_{i-1} + S_1\bar{S}_0\,Q_{i+1} + S_1S_0\,\text{Data}_i$$

Bu fonksiyon 4 AND kapısı ve çıkışlarını toplayan bir OR kapısıyla tasarlanabilir, ama 4'ten 1'e bir multiplexer kullanmak daha pratiktir. Multiplexer'ın iki seçme girişi vardır ve bunların durumuna göre dört girişinden hangisinin çıkışa aktarılacağına karar verir. Tablonun son sütunu hangi girişe neyin bağlandığını gösterir.

Standart grafik sembolünde mod seçimi için iki giriş bulunur. Sağa ve sola kaydırmada dışarıdan gelecek değerler için sol seri giriş (left serial input) ve sağ seri giriş (right serial input) uçları vardır.

Soru: 4 bit için 4 multiplexer gerekir, değil mi? Q_{i-1} de başka bir multiplexer'ın sonucu oluyor. **Cevap:** Evet, her D flip-flop'un önüne bir multiplexer eklemek gerekir.

> **Hocanın vurgusu:** Farklı bir fonksiyon tablosu verildiğinde, bunu gerçekleştiren devreyi bir multiplexer çizerek tasarlayabilmelisiniz.

4. Saklayıcı Transfer İşlemleri ve Mikro İşlemler

Hoca bu konuyu sonraki hafta ayrıntılı anlatacağını, burada hızlıca geçeceğini söyledi. Çünkü CPU içindeki veri yolu (data bus) ve adres yolu (address bus) yapıları kurulurken saklayıcılar arası transfer işlemlerinden yararlanılacaktır.

4.1 Datapath ve Control Unit

CPU'yu iki temel blok oluşturur:

- **Datapath unit:** Veriyi işleyen ve saklayıcıları tutan birimdir. İçinde aritmetik lojik birim, kaydırma birimi (shift unit) ve saklayıcılar bulunur. Control unit'ten gelen işaretlere göre tanımlı işlemi bunlar üzerinde gerçekleştirir.
- **Control unit:** Dışarıdan tanımlanan komutları (instruction) ve bu komutları gerçekleştirmek için gereken kontrol işaretlerini üreten birimdir. Kontrol işaretlerini datapath unit'e gönderir.

İşlem sonucunda oluşan durum bitleri ise datapath unit'ten control unit'e gönderilir. Bunlar overflow, zero, negative ve gerekiyorsa carry bitleridir. Dersin bundan sonraki bölümünde birkaç hafta boyunca datapath unit'in nasıl oluşturulacağı konuşulacaktır: içindeki aritmetik lojik

birim tasarlanacak, işlem yapılacak operandlar saklayıcılar üzerinden seçilecek ve saklayıcılar arasında veri transferi yapılacaktır.

4.2 Notasyon

Bu derste belirli bir işlemciye özgü tasarım yapılmaz. Ne 8086 ne de Motorola ailesinden işlemciler anlatılır. Anlatım genel olduğu için notasyon da geneldir:

- **Harf ve numara:** Saklayıcıları gösterir. Adres saklayıcısı, R_2, hedef saklayıcı (destination register), komut saklayıcısı (instruction register) gibi.
- **Bit sırası:** Saklayıcı büyük R harfiyle gösterilir. 8 bitlik bir saklayıcıda sağda en düşük anlamlı, solda en yüksek anlamlı bit bulunur.
- **Parantez:** Saklayıcının belli bir bölümünü gösterir. Örneğin 16 bitlik AX saklayıcısı alt ve üst parça (low part, high part) olarak iki 8 bitlik saklayıcı gibi ya da bir bütün olarak kullanılabilir.
- **Ok:** Transfer ve atama işlemini gösterir: $R_1 \leftarrow R_2$, yani R_1'e R_2'nin içeriğini yükle.
- **Virgül:** Aynı satırda virgülle ayrılan işlemler aynı anda yapılan transferlerdir (simultaneous transfers).
- **Köşeli parantez:** Bellek adresini gösterir.

4.3 Koşullu Transfer

Koşullu transfer (conditional transfer) örneğinde R_1'den R_2'ye bir yükleme yapılacaktır, ama bunun için K_1 kontrol ucunun 1 olması gerekir:

$$K_1 : R_2 \leftarrow R_1$$

K_1 aslında R_2 saklayıcısının load enable ucudur. Az önce tasarlanan yapıda load ucu 1 iken saklayıcıya yeni değer yükleniyordu, burada da $K_1 = 1$ iken R_2'ye R_1'in içeriği yüklenir.

4.4 Mikro İşlem

Mikro işlem (micro operation), saklayıcılarda veya bellekte duran veri üzerinde yapılan en temel (elementary) işlemdir. Assembly'de öğrenilen bir komut (instruction) değildir. Bazı komutlar 3 clock periyodunda, bazıları 6 clock periyodunda gerçekleşir. O 6 periyodun her birinde yapılan en temel işlem bir mikro işlemdir. Bir dizi mikro işlem bir araya gelerek bir komutu oluşturur, ama bazen tek bir mikro işlem de tek başına bir komut olabilir.

Mikro işlemler şöyle ayrılır:

- **Transfer mikro işlemleri.**
- **Aritmetik mikro işlemler:** Slaytta toplama, çıkarma, çarpma, bölme yer alır. Hoca birler ve ikiler tümleyeni (one's, two's complement) işlemlerini de buraya saydı, çünkü çıkarma ikiler tümleyeniyle yapılır. Artırma ve azaltma da bu gruptadır.
- **Lojik mikro işlemler:** NOT, AND, OR, XOR.
- **Kaydırma mikro işlemleri:** Shift left ve shift right.

Slayttaki örnekler arasında iki saklayıcıyı toplayıp sonucu R_1'e atayan bir toplama, R_1'den R_6'yı çıkaran bir çıkarma ve bir XOR işlemi vardır. Mikro işlemler de koşula bağlanabilir: Bir örnekte, sol taraftaki K_1, K_2 koşulu sağlanmadan OR işleminin sonucu R_1'e atanmaz.

Hoca, bu derste tasarımların kâğıt üzerinde yapıldığını, ama bu işlemler için VHDL ve Verilog gibi donanım tanımlama dillerinin (hardware description language, HDL) bulunduğunu ve gömülü tasarımda çok kullanıldığını da ekledi.

4.5 Toplayıcı-Çıkarıcı ile Koşullu Aritmetik

Daha önce görülen toplayıcı-çıkarıcı (adder-subtractor) devresi belli bir koşul altında kullanılabilir:

$$\bar{X}K_1 : R_1 \leftarrow R_1 + R_2$$

$$XK_1 : R_1 \leftarrow R_1 - R_2$$

Çıkarma ikiler tümleyeniyle yapıldığı için $R_1 - R_2 = R_1 + \overline{R_2} + 1$ yazılabilir (ara adım eklendi). İki işlemde de hedef saklayıcı R_1'dir ve toplayıcı-çıkarıcının çıkışı R_1'e yüklenecek şekilde bağlıdır. R_1'in load enable ucu K_1 ile kontrol edilir. İşlem sonucunun R_1'e yüklenebilmesi için $K_1 = 1$ olmalıdır, koşullardaki K_1 budur.

Toplayıcı-çıkarıcı, n tam toplayıcının (full adder) bir araya getirilmesiyle kurulur. R_2 girişleri bir XOR ile kontrol edilir, bu XOR'un kontrol girişine slaytta X denmiştir:

- $X = 0$ iken devre toplama yapar. XOR'un bir girişi 0 olunca diğer giriş çıkışa doğrudan aktarılır.
- $X = 1$ iken devre çıkarma yapar. XOR'un bir girişi 1 olunca çıkış diğer girişin tümleyeni olur.

  Ders sonrası not: Hoca çıkarmayı anlatırken "X girişinin komplementini alıp" dedi. Burada X adı hem toplayıcının bir girişi için hem de kontrol ucu için kullanıldı. Tümlenen, kontrol ucu değil R_2'nin bitleridir.

Durum bitleri şöyle oluşturulur:

- **Carry (C):** En yüksek anlamlı bitten dışarı çıkan elde bitidir. Bir bitlik bir saklayıcıda tutulur.
- **Overflow (V):** İşaret bitine gelen elde (C_{n-1}) ile işaret bitinden dışarı çıkan elde (C_n) arasında XOR yapılır: $V = C_n \oplus C_{n-1}$. Overflow varsa bu bit 1 olur.

Aritmetik birimin içinde de bir toplama-çıkarma devresi vardır. Aritmetik ve lojik işlemleri yapan blok bu eleman kullanılarak tasarlanacaktır.

4.6 Seri Transfer

Slayttaki seri transfer örneğinde iki 4 bitlik kaydırmalı saklayıcı arka arkaya bağlanarak (birinin çıkışı diğerinin seri girişine) 8 bitlik bir yapı oluşturulmuştur. Burada clock girişinde yine gated clock vardır. Hocaya göre tek başına çalışan bir devrede bu kullanılabilir, ama işlemcinin içinde tercih edilmez.

5. Seri Toplayıcı

Hoca ilginç bulduğu bir örnek olarak seri toplayıcıyı (serial adder) anlattı.

5.1 Yapı

- İki adet 4 bitlik kaydırmalı saklayıcı vardır: Register A ve Register B. Dışarıdan sayı girişi yalnız Register B'nin seri girişinden yapılabilir.
- Tek bir tam toplayıcı kullanılır. Saklayıcıların seri çıkışları bu toplayıcının X ve Y girişlerine bağlıdır.
- Toplayıcının elde biti (carry) bir D flip-flop'ta tutulur ve varsa bir sonraki toplamaya aktarılır.
- Toplam (sum) çıkışı doğrudan Register A'nın seri girişine bağlıdır, yani toplama sonucu Register A'da toplanır.
- Bütün clock girişleri bir noktada birleşir. Burada da gated clock vardır, ama hocaya göre bu önemli değildir, yapı bağımsız çalışan bir devre gibi düşünülebilir.
- Bütün saklayıcıların reset girişi vardır, devre başta resetlenir.

5.2 Soru: Kaç Clock Periyodu Gerekir?

Her iki saklayıcı ve elde flip-flop'u sıfırlanmıştır. Elimizde iki 4 bitlik sayı vardır: A sayısı ve B sayısı. Sayıların saklayıcılara yüklenmesi de işin içindedir. Bu iki sayı toplanıp sonuç Register A'da tutulana kadar kaç clock periyodu geçer?

Öğrencilerden sırayla 9, 8 ve 16 cevapları geldi. Hoca sayıların henüz saklayıcılarda olmadığını ve yüklemenin de süreye dahil olduğunu hatırlattı. Toplayıcının çıkışlarının 4 bit olup olmadığı

da soruldu. Cevap, seri çıkışların 1 bit olduğu ve toplayıcının yalnız bu iki biti topladığıydı. Doğru cevap **12 clock periyodudur**:

Clock	Register B'ye giren	Register B'den çıkan	Register A'dan çıkan	Register A'ya yazılan
1-4	A sayısı	0	0	0
5-8	B sayısı	A sayısı	0	A sayısı
9-12		B sayısı	A sayısı	A + B

Adımlar şöyledir:

- **İlk 4 clock:** Tek seri giriş kanalı Register B olduğu için A sayısı Register B'ye yüklenir. Bu 4 bitlik yükleme 4 clock sürer. O sırada toplayıcı yine toplama yapar, $0 + 0$ sonucu Register A'ya yazılır.
- **Sonraki 4 clock:** B sayısı Register B'ye yüklenir. Bu sırada A sayısı Register B'den en düşük anlamlı bitten başlayarak dışarı atılır: önce A_0, sonraki clock'ta A_1, ve böyle devam eder. Bu bitler Register A'daki sıfırlarla toplanır ve toplam Register A'ya yazılır. Yani aslında iki kere toplama yapılmış olur: önce sıfırla, sonra birbirleriyle. Sıfırla toplamada elde oluşmadığı için elde flip-flop'u 0 kalır (ara adım eklendi). 8. periyodun sonunda A sayısının bütün bitleri Register A'ya taşınmış olur.
- **Son 4 clock:** İki saklayıcının içindeki sayılar bit bit toplanır ve sonuç Register A'da oluşur.

  Ders sonrası not: Hoca 8. periyoda kadarki adımı anlatırken A sayısının "register B'de bulunan" 4 sıfırla toplandığını söyledi. Doğrusu, sıfırlar Register A'dadır. Hoca bunu kısa süre önce de "register A'daki 0 ile register B'de yüklediğin A sayısı" diye doğru anlatmıştı. Bütün 4 bitlik sayı çiftleri için yapılan benzetimde 12. clock sonunda toplam Register A ve elde flip-flop'unda doğru çıktı.

Soru: Başta da sıfırla toplamıyor muyuz, orada da bir süre geçmesi gerekmez mi? **Cevap:** Sıfırla toplama ayrı bir süre almaz. Sayı yüklenirken zaten clock gelir ve her clock'ta toplama yapılır, sonuç Register A'ya yüklenir. B sayısını yüklemek için harcanan 4 clock, A sayısını Register A'ya taşımak için de kullanılır.

5.3 Avantaj ve Dezavantaj

Zaman kısıtı yoksa, tek bir tam toplayıcıyla devre basit tutulabilir. Bedeli ise işlemlerin daha geç gerçekleşmesi, daha uzun sürmesidir.

Soru: Tam toplayıcılar clock'tan bağımsız mı çalışıyor? Clock olmadığında ne yapıyorlar, bekliyorlar mı? **Cevap:** Tam toplayıcı kombinasyonel bir devredir ve clock'a ihtiyacı yoktur. Kombinasyonel devrenin girişinde sürekli bir işaret vardır ve içerideki devre buna göre her zaman bir sonuç üretir. X, Y, Z girişleri clock periyodu boyunca değişmez, sonuç toplam çıkışında görünür. Clock yalnız saklayıcılara yeni değer yüklerken gerekir: Toplam çıkışı flip-flop girişinde bekler, yükleme clock pozitif kenar gördüğünde yapılır.

6. Asenkron (Ripple) Sayıcılar

Sayıcılarla ilgili iki temel tasarım vardır: ripple sayıcı (diğer adıyla asenkron sayıcı) ve senkron sayıcı. Sayıcılar da tasarımlarda kullanılacak birer yapı taşıdır.

> **Hocanın vurgusu:** Ripple, yani asenkron sayıcıları bilin. Yapı asenkron olduğu için çok tercih edilmez. Esas tasarımlar her zaman senkron sayıcılarla olacaktır.

Asenkron sayıcı D, T ve JK flip-flop'ların hepsiyle yapılabilir.

6.1 D Flip-Flop ile Ripple Sayıcı

Slayttaki örnekte iki D flip-flop vardır. Çıkışları A ve B diye adlandırılmıştır. Hoca bunları ağırlık sırasına göre A_0 ve A_1 gibi düşünmemizi istedi: A düşük, B yüksek anlamlı bittir. Bağlantılar şöyledir:

- Dışarıdan gelen clock, düşük anlamlı bitin clock girişine uygulanır.
- Flip-flop'un $\bar{Q}$ çıkışı kendi D girişine bağlanır. Böylece her tetiklemede içeriğinin tersini alır.
- Aynı $\bar{Q}$ çıkışı, bir yüksek anlamlı bitin clock girişine bağlanır.
- Bütün flip-flop'larda öncelikli bir temizleme (clear, reset) ucu vardır.

Çalışma mantığı zaman diyagramından görülür. Flip-flop'lar pozitif kenar tetiklemelidir. A çıkışı clock'un her pozitif kenarında durum değiştirir: 0'dan 1'e çıkar, sonraki pozitif kenarda 1'den 0'a iner. Alttaki flip-flop'un clock'u $\bar{A}$'dır. Bu yüzden diyagramda A'nın tersi çizilir ve B, bu işaretin pozitif kenarlarında durum değiştirir.

BA çıkışları sırayla okunduğunda 00, 01, 10, 11 görülür, sonra ikisi de 0'a düşer ve aynı döngü başlar. Yani 0, 1, 2, 3 diye yukarı sayılır: devre bir yukarı sayıcıdır (up counter).

Flip-flop'ların clock girişleri ortak tek bir clock ile beslenmediği için bu yapıya asenkron sayıcı denir. Bir flip-flop'un çıkışının diğerinin clock girişine bağlanması senkron yapıyı bozar.

6.2 T Flip-Flop ile 4 Bitlik Ripple Sayıcı

Hoca D flip-flop'lu örnekten daha kolay anlaşılacağını düşündüğü bir tasarımı tahtada çizdi. T flip-flop'un davranışı şöyledir:

T	$Q(t+1)$	Anlamı
0	$Q(t)$	Önceki durum korunur
1	$\overline{Q(t)}$	Önceki durumun tümleyeni alınır

Tasarım şöyledir:

- Bütün T girişleri 1'e bağlanır.
- Dış clock, en düşük anlamlı bitin (Q_0) flip-flop'una girer.
- Her flip-flop'un Q çıkışı, bir yüksek anlamlı bitin clock girişine bağlanır.
- Flip-flop'lar önce negatif kenar tetiklemeli (negative edge triggered) seçilir. Düşen kenar, negatif kenar demektir.

Çizimde en solda en düşük anlamlı bit Q_0, en sağda en yüksek anlamlı bit Q_3 bulunur. Zaman diyagramı şöyle çizilir:

- $T = 1$ olduğu için Q_0, clock'un her düşen kenarında tümlenir.
- Q_0 bir sonraki flip-flop'un clock'udur. Q_1, Q_0'ın her düşen kenarında tümlenir.
- Q_2, Q_1'in; Q_3, Q_2'nin düşen kenarlarında aynı şekilde tümlenir.

Her clock periyodunda çıkışlar $Q_3Q_2Q_1Q_0$ olarak okunur: 0000, 0001, 0010, 0011, … Hocanın diyagramdan okuduğu değerler arasında 0101 de vardır, yani 5. Böylece yukarı sayan bir sayıcı elde edilmiştir.

Hoca bu devrede tek değişikliğin bile yönü çevirdiğini söyledi: Negatif kenar yerine pozitif kenar tetiklemeli flip-flop kullanılırsa devre aşağı sayıcı (down counter) olur. Zaman diyagramını çizip denememizi istedi.

> **Ek bilgi (derste anlatılmadı):** Hocanın istediği denemenin sonucu: Pozitif kenar tetiklemeli hâlde 0000'dan başlayan devre 0000, 1111, 1110, 1101, ..., 0001, 0000 diye aşağı sayar. Çünkü bir üst bit, alt bitin 0'dan 1'e çıktığı anda tümlenir. Yön, kenarla birlikte bir üst bitin clock'una Q mu $\bar{Q}$ mu bağlandığına da bağlıdır. Bölüm 6.1'deki gibi $\bar{Q}$ bağlı pozitif kenarlı yapı yukarı, Q bağlı pozitif kenarlı yapı aşağı sayar.

6.3 D ve JK ile Alternatifler

T flip-flop burada yalnız bir görev için kullanılmıştır: sonraki durumda hep mevcut durumun tümleyenini almak. Aynı davranış diğer flip-flop'larla da elde edilir:

- **D flip-flop:** $\bar{Q}$ çıkışı D girişine bağlanır.

- **JK flip-flop:** $J = K = 1$ yapılır. JK karakteristik tablosunda bu giriş, $\overline{Q(t)}$ satırıdır.

Clock'un pozitif ya da negatif kenarının kullanılmasına göre yukarı veya aşağı sayma sağlanır. Hoca bu çözümlerle biraz egzersiz yapılmasını tavsiye etti.

7. Senkron İkili Sayıcı Tasarımı

Senkron sayıcı, tercih edilen ikinci biçimdir. Slayttaki sayma dizisi tablosunda yukarı sayma (upward counting) ve aşağı sayma (downward counting) için ardışık değerler verilmiştir. Hoca tasarımı boş bir sayfada öğrencilerle birlikte yaptı: 4 bitlik yukarı sayıcı, D flip-flop ile.

7.1 Durum Tablosu

Durum tablosuna mevcut durum ($A_3A_2A_1A_0$) ve sonraki durum yazılır. 4 bit için 16 kombinasyon vardır. Sayıcı olduğu için her satırın sonraki durumu o değerin bir fazlasıdır. Son satırda, 15'ten sonra 0'a dönülmesi istendiği için 0000 yazılır.

D flip-flop kullanıldığında sonraki durum aynı zamanda flip-flop girişidir, bu yüzden ayrıca giriş sütunu yazılmaz. Karnaugh haritaları sonraki durum sütunlarından çıkarılır.

7.2 Karnaugh Haritaları

Haritalar en düşük anlamlı bitten başlanarak kurulur. Satırlarda A_3A_2, sütunlarda A_1A_0 bulunur ve sıra 00, 01, 11, 10'dur. Hoca değerleri durum tablosundaki sırayla okudu: A_0 için her dörtlükte 1, 0, 1, 0 ve A_1 için 0, 1, 1, 0. Bu değerler harita sırasına yerleştirildiğinde bütün satırlar aynı çıkar (ara adım eklendi):

A_1A_0	00	01	11	10
D_{A_0} (her satırda)	1	0	0	1
D_{A_1} (her satırda)	0	1	0	1

D_{A_0} için 1'ler sekizli bir grup oluşturur. Bu grubun terimi:

$$D_{A_0} = \bar{A}_0$$

D_{A_1} için 1'ler iki dörtlü grup oluşturur.

Soru: Bu iki grubun terimleri nedir? Bir öğrenci doğrudan "A_0 XOR A_1" dedi. **Cevap:** Hoca bunun çok kestirme olduğunu, önce açık hâlinin söylenmesini istedi: $\bar{A}_1A_0 + A_1\bar{A}_0$. Bu ifadenin XOR olduğu buradan görülür:

$$D_{A_1} = \bar{A}_1A_0 + A_1\bar{A}_0 = A_1 \oplus A_0$$

Aynı işlem diğer bitler için yapıldığında:

$$D_{A_2} = A_2 \oplus A_1A_0$$

$$D_{A_3} = A_3 \oplus A_2A_1A_0$$

Hoca buradaki kalıba dikkat çekti: Bu ilişki görüldükten sonra tasarım 4 bitten 5, 6, 7 bite genişletilebilir, çünkü denklemin nasıl formülize edildiği görülür.

> **Ek bilgi (derste anlatılmadı):** Kalıbın genel hâli $D_{A_i} = A_i \oplus A_{i-1}A_{i-2}\cdots A_0$ biçimindedir. Yani bir bit, altındaki bütün bitler 1 iken tümlenir. Bu genel denklem 5, 6 ve 7 bit için bütün durumlarda hesapla denendi ve doğru çıktı.

Slaytta bu tasarımın devre şeması ve standart grafik sembolü de verilir.

8. Load Girişli Sayıcıdan Başka Sayıcılar Elde Etmek

Slayttaki "other counters" bölümüne göre aynı mantıkla aşağı sayıcı da kurulabilir. Modulo n (divide by n) biçiminde sayan bir sayıcı da oluşturulabilir: Örneğin 0'dan 7'ye sayan bir sayıcı ya da 0-9 arasında sayan modulo 10 sayıcı, yani BCD sayıcı. Hoca asıl göstermek istediği şeyin, elimizdeki ikili sayıcıyı farklı biçimlerde sayan bir sayıcıya dönüştürmek olduğunu söyledi.

Kullanılan blok, load ve count uçları eklenmiş senkron ikili sayıcıdır. Count girişi sürekli 1'dir.

8.1 BCD (Modulo 10) Sayıcı

BCD sayıcı 0'dan başlar ve en fazla 9 değerini alır. Görülebilecek en büyük sayı olan 9 görüldüğünde sayıcının yeni değer yüklemesi sağlanmalıdır.

- 9, ikili olarak 1001'dir. Q_3 ve Q_0 çıkışları bir AND kapısına verilir: Load $= Q_3Q_0$.
- Bu iki giriş 1 iken AND çıkışı 1 olur ve load girişi 1 olur.
- Paralel veri girişlerine 0000 verilir.

Böylece sayıcı 0, 1, …, 9, 0, … diye sayar. Bu dizi hesapla doğrulandı.

8.2 9-14 Arası Sayan Sayıcı

Slayttaki ikinci örnek bir modulo 6 sayıcıdır. Sıfırlamada 9 ile başlar (preset 9 on reset), son değer olan 14'te yine 9 yükler (load 9 on terminal count 14). Sayıcı 9'dan başlayıp 14'e kadar sayar, 14'ten sonra 9'u görür.

- En büyük değer 14'tür, ikili olarak 1110. 14'ü görmek, en yüksek anlamlı 3 bitin 1 olduğunu görmek demektir, çünkü sayıcı 15'i zaten saymayacaktır. Bu nedenle Load $= Q_3Q_2Q_1$ yeterlidir.
- Yüklenecek değer 9'dur, yani paralel girişlere 1001 verilir.

Sayıcı 9, 10, 11, 12, 13, 14, 9, … diye sayar. Altı değer olduğu için modulo 6 adını alır. Bu dizi de hesapla doğrulandı.

> **Hocanın vurgusu:** 0-15 arası sayan bir ikili sayıcıyı 0-9 arası sayan BCD sayıcıya ya da 9-14 arası sayan sayıcıya nasıl dönüştüreceğinizi çalışın. Bu çalışma sonraki konuları daha iyi takip etmenizi sağlayacaktır.

9. Rastgele Sayma Düzenli Sayıcı ve JK Geçiş Tablosu

Son örnekte sayma düzeni rastgeledir (arbitrary count sequence) ve kullanılmayan durumlar (unused states) vardır. Tasarımda farklılık olması için JK flip-flop kullanılır. Sayma düzeni şöyledir:

$$0 \to 1 \to 3 \to 2 \to 4 \to 6 \to 0 \to \cdots$$

> **Ders sonrası not:** Altyazıda hocanın bu örneğe girerken "2, 4, 6, 8 kullanılmayan state'ler" dediği geçiyor. Bu düzende kullanılmayan durumlar 5 ve 7'dir. Hoca da tabloyu doldururken don't care'i bu iki satıra yazdı.

9.1 Durum Tablosu

En büyük değer 6'dır ve 3 bitle ifade edilir. Bu yüzden 3 durum değişkeni gerekir: A_2, A_1, A_0. Tabloya 8 kombinasyonun hepsi yazılır. D flip-flop'ta sonraki durum aynı zamanda flip-flop girişi olduğu için giriş sütunu yazılmıyordu. JK ile tasarımda ise bu sütunlar ayrıca yazılmalıdır.

Sonraki durum, mevcut değerin sayma düzenindeki ardılıdır: 0'dan 1'e, 1'den 3'e, 2'den 4'e, 3'ten 2'ye, 4'ten 6'ya ve 6'dan yeniden 0'a geçilir. Hoca 5 satırına gelince bir an duraksadı, bir öğrenci "5 yok, 6'dan tekrar 0'a geçeceğiz" dedi. 5 ve 7 tanımsız durumlar olduğu için sonraki durumlarına don't care yazılır.

9.2 JK Geçiş Tablosu

Flip-flop girişlerini yazarken şu soru sorulur: Mevcut durum 0 iken sonraki durumun 0 olması için J ve K'ya ne uygulanmalı? Bunu cevaplamak için JK flip-flop'un doğruluk tablosunu ve geçiş tablosunu (excitation table) bilmek gerekir.

JK karakteristik tablosu:

J	K	$Q(t+1)$	Anlamı
0	0	$Q(t)$	Değişmez
0	1	0	Reset
1	0	1	Set
1	1	$\overline{Q(t)}$	Tümleme

> **Hocanın vurgusu:** Geçiş tablosunu ezbere bilmenize gerek yok, karakteristik tablodan kendiniz üretebilmelisiniz.

Geçiş tablosu şöyle üretilir:

- $0 \to 0$**:** Sonraki durum mevcut durumla aynı olsun ya da 0 olsun. Bunu "değişmez" (00) ve "reset" (01) satırları sağlar. İkisinde de $J = 0$, K önemsizdir.
- $0 \to 1$**:** Sonraki durum 1 olsun ya da mevcut durumun tümleyeni olsun. Bunu "set" (10) ve "tümleme" (11) satırları sağlar. $J = 1$, K önemsizdir.
- $1 \to 0$**:** Sonraki durum mevcut durumun tümleyeni olsun ya da 0 olsun. Bunu "reset" (01) ve "tümleme" (11) satırları sağlar. J önemsiz, $K = 1$.
- $1 \to 1$**:** Sonraki durum 1 olsun ya da mevcut durumla aynı olsun. Bunu "değişmez" (00) ve "set" (10) satırları sağlar. J önemsiz, $K = 0$.

$Q(t)$	$Q(t+1)$	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

9.3 Flip-Flop Girişlerinin Doldurulması

Tablonun sütunları geçiş tablosuna bakılarak bit bit doldurulur. Örneğin A_2 için mevcut durum 0 ve sonraki durum 0 ise $J_2K_2 = 0X$ yazılır. Mevcut durum 0 ve sonraki durum 1 ise $1X$ yazılır. Sonraki durumu don't care olan satırlarda her iki giriş de don't care olur.

$A_2A_1A_0$	Sonraki	J_2K_2	J_1K_1	J_0K_0
000	001	0X	0X	1X
001	011	0X	1X	X0
010	100	1X	X1	0X
011	010	0X	X0	X1
100	110	X0	1X	0X
101	XXX	XX	XX	XX
110	000	X1	X1	0X
111	XXX	XX	XX	XX

Hoca J_0K_0 sütununu 101 satırına kadar okudu, son iki satır aynı kuralla eklendi (ara adım eklendi). Tablonun bütün satırları hesapla doğrulandı.

Flip-flop girişleri hazır olduğuna göre bir sonraki adım bunlardan sadeleştirilmiş fonksiyonları çıkarmaktır. Süre dolduğu için ders burada bitti.

> **Hocanın vurgusu:** Bu örneğin sonunda kullanılmayan durumlar için de bir değerlendirme yapmak gerekir: Devre kullanılmayan bir durumdan başlarsa sayma işlemine nasıl devam eder? Bunu dikkate almak gerekir.

Ek bilgi (derste anlatılmadı): Derste tamamlanmayan sadeleştirme, tablodaki don't care'lerle en az terimli şu sonucu verir: $J_2 = A_1\bar{A}_0$, $K_2 = A_1$, $J_1 = A_0 + A_2$, $K_1 = \bar{A}_0$, $J_0 = \bar{A}_2\bar{A}_1$, $K_0 = A_1$. Bu denklemlerle kullanılmayan durumlar $5 \to 7 \to 2$ ve $7 \to 2$ biçiminde ilerler. Devre 5'ten başlarsa iki clock, 7'den başlarsa bir clock sonra sayma düzenine girer. Hesap, sayma düzenindeki bütün geçişler için de denendi.

Dersin Toparlaması

Ders, ardışıl devre tasarımının temel adımlarını bir dizi tanıyıcıyla tekrar ettikten sonra işlemcinin yapı taşlarına geçti. Saklayıcılara load girişi eklemenin iki yolu karşılaştırıldı: clock gating clock'u geciktirdiği için işlemci içinde kullanılmaz, load-controlled feedback ise clock'u bütün flip-flop'lara doğrudan bağlı bıraktığı için tercih edilir. Fonksiyon tablosundan flip-flop giriş denklemi yazmak ve bunu multiplexer ile gerçekleştirmek gösterildi. Saklayıcı transfer notasyonu, koşullu transfer ve mikro işlem türleri, CPU'daki datapath unit ve control unit ayrımı üzerinden hızlıca tanıtıldı. Seri toplayıcı örneğinde tek tam toplayıcıyla 4 bitlik iki sayının yüklenip toplanmasının 12 clock sürdüğü görüldü.

Sayıcılarda asenkron yapının kenar seçimine göre yukarı veya aşağı saydığı, ama esas tasarımların senkron olacağı anlatıldı. D flip-flop'lu ikili sayıcının denklemleri çıkarıldı, load girişiyle BCD ve 9-14 arası sayıcılar elde edildi. JK flip-flop ile rastgele sayma düzenli sayıcının durum tablosu ve giriş sütunları doldurduldu. Hoca bir sonraki derste bu örneği tamamlayabileceğini ya da benzer bir örnek daha yapacağını, kullanılmayan durumlardan başlama değerlendirmesinin de yapılması gerektiğini söyledi. Hoca ayrıca veri yolu yapısına ve datapath unit tasarımına önümüzdeki haftalarda ayrıntılı dönüleceğini belirtti.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.